

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TP_Informatique_ENS_1ère_année_AN_05.pdf

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

At the end of this lab, the student is able to:

- Manipulate lists (creation, addition, deletion, traversal)
- Use list methods: `append()`, `remove()`, `sort()`, `index()`
- Understand indexing and slicing
- Implement simple search and sort algorithms
- Distinguish between list and tuple
- Use list comprehensions to filter data

Lab Plan

Phase	Duration	Activity
Welcome and review	5 min	Recap on lists
Exercise 1	15 min	List manipulation
Exercise 2	15 min	Searching in a list
Exercise 3	20 min	List of grades with coefficients
Exercise 4	20 min	Bubble sort
Exercise 5	15 min	Filtering a list
MCQ and wrap-up	15 min	Self-assessment and conclusion

Exercise 1 — List Manipulation (15 min)

Problem statement: Create a script `listes.py` that:

1. Creates an empty list `notes`
2. Adds 5 grades entered by the user
3. Displays the list
4. Displays the max, min and average grade
5. Removes the lowest grade
6. Displays the sorted list

Expected example:

```
Note 1 : 12
Note 2 : 15
Note 3 : 8
Note 4 : 19
Note 5 : 10
List : [12.0, 15.0, 8.0, 19.0, 10.0]
Max : 19.0 – Min : 8.0 – Average : 12.80
After removing the lowest: [10.0, 12.0, 15.0, 19.0]
```

Solution:

```
notes = []
for i in range(5):
    n = float(input(f"Grade {i+1} : "))
    notes.append(n)

print(f"List : {notes}")
print(f"Max : {max(notes)} – Min : {min(notes)} – Average : {sum(notes)/len(notes):.2f}")

notes.remove(min(notes))
notes.sort()
print(f"After removing the lowest: {notes}")
```

Exercise 2 — Searching in a List (15 min)

Problem statement: Create a script `recherche.py` that:

1. Creates a list of first names
2. Asks the user for a first name
3. Indicates whether the first name is in the list and at which position
4. Offers to add the first name if it does not exist

Expected example:

```
Current list : ['Ali', 'Sara', 'Mohamed', 'Leila', 'Ahmed']
First name to search: Sara
Sara found at position 2.
```

```
flowchart TD
    A[Start] --> B[Enter first name]
    B --> C{First name in list?}
    C -->|Yes| D[Display position]
    C -->|No| E{Add?}
    E -->|Yes| F[Add and sort]
    E -->|No| G[End]
    D --> G
    F --> G
```

Solution:

```
prenoms = ["Ali", "Sara", "Mohamed", "Leila", "Ahmed"]
print(f"Current list : {prenoms}")

recherche = input("First name to search: ").capitalize()

if recherche in prenoms:
    pos = prenoms.index(recherche) + 1
    print(f"{recherche} found at position {pos}.")
else:
    print(f"{recherche} is not in the list.")
```

```
ajout = input("Would you like to add it? (y/n) : ")
if ajout == "y":
    prenom.append(recherche)
    prenom.sort()
    print(f"New list : {prenom}")
```

Exercise 3 — List of Grades with Coefficients (20 min)

Problem statement: Create a script `notes_coef.py` that:

1. Asks for the number of subjects
2. For each subject, asks for the name, grade and coefficient
3. Stores the data in a list of dictionaries
4. Calculates the weighted average
5. Displays a summary table

Expected example:

```
Number of subjects: 3

--- Subject 1 ---
Name: Maths
Grade (0-20): 14
Coefficient: 4

--- Subject 2 ---
Name: French
Grade (0-20): 12
Coefficient: 3

--- Subject 3 ---
Name: English
Grade (0-20): 16
Coefficient: 2

Weighted average: 13.78/20
```

Solution:

```
matieres = []
nb = int(input("Number of subjects: "))

for i in range(nb):
    print(f"\n--- Subject {i+1} ---")
    nom = input("Name: ")
    note = float(input("Grade (0-20): "))
    coef = float(input("Coefficient: "))
    matieres.append({"nom": nom, "note": note, "coef": coef})

total_points = sum(m["note"] * m["coef"] for m in matieres)
total_coefs = sum(m["coef"] for m in matieres)
moyenne = total_points / total_coefs if total_coefs > 0 else 0

print("\n" + "="*45)
print(f"{'Subject':<15} {'Grade':<8} {'Coef':<8} {'Total':<8}")
print("="*45)
for m in matieres:
    total = m["note"] * m["coef"]
    print(f"{'m['nom']':<15} {'m['note']':<8.2f} {'m['coef']':<8.2f} {'total':<8.2f}")
print("="*45)
```

```
print(f"Weighted average: {moyenne:.2f}/20")
```

Exercise 4 — Bubble Sort (20 min)

Problem statement: Create a script `tri_bulles.py` that implements the bubble sort algorithm without using `sort()`. The program must:

1. Create a random list of 10 numbers
2. Display the original list
3. Sort using the bubble sort algorithm
4. Display the sorted list
5. Count the number of swaps performed

```
flowchart TD
    A[Unsorted list] --> B[i from 0 to n-1 ]
    B --> C[j from 0 to n-i-2 ]
    C --> D{ t[j] > t[j+1] ? }
    D -->|Yes| E[Swap t[j] ↔ t[j+1]]
    D -->|No| F[Skip]
    E --> G[ next j ]
    F --> G
    G --> H{ End of pass? }
    H -->|No| C
    H -->|Yes| I[ next i ]
    I --> J{ List sorted? }
    J -->|No| B
    J -->|Yes| K[Sorted list]
```

Solution:

```
import random

def tri_bulles(liste):
    """Bubble sort, returns the number of swaps."""
    n = len(liste)
    echanges = 0
    for i in range(n):
        for j in range(0, n - i - 1):
            if liste[j] > liste[j + 1]:
                liste[j], liste[j + 1] = liste[j + 1], liste[j]
                echanges += 1
    return echanges

liste = [random.randint(1, 100) for _ in range(10)]
print(f"Original: {liste}")

echanges = tri_bulles(liste)
print(f"Sorted : {liste}")
print(f"Swaps : {echanges}")
```

Exercise 5 — Filtering a List (15 min)

Problem statement: Create a script `filtre.py` that:

1. Creates a list of 20 random numbers between 0 and 100
2. Creates a new list containing only even numbers

3. Creates a new list containing only numbers ≥ 50
4. Displays the three lists with their lengths

Expected example:

```
All : [12, 45, 78, 23, 56, 91, 34, 67, 8, 90, ...] (20 elements)
Even : [12, 78, 56, 34, 8, 90, ...] (11 elements)
 $\geq 50$  : [78, 56, 91, 67, 90, ...] (8 elements)
```

Solution:

```
import random

nombres = [random.randint(0, 100) for _ in range(20)]
pairs = [n for n in nombres if n % 2 == 0]
grands = [n for n in nombres if n >= 50]

print(f"All : {nombres} ({len(nombres)} elements)")
print(f"Even : {pairs} ({len(pairs)} elements)")
print(f" $\geq 50$  : {grands} ({len(grands)} elements)")
```

Self-Assessment MCQ

1. Which method adds an element to the end of a list?
 - a) `insert()` ✓
 - b) `append()`
 - c) `add()`
2. What is the complexity of bubble sort in the worst case?
 - a) $O(n)$
 - b) $O(n^2)$ ✓
 - c) $O(\log n)$
3. Which index corresponds to the first element of a Python list?
 - a) 1
 - b) 0 ✓
 - c) -1
4. How to remove the first occurrence of a value from a list?
 - a) `del()`
 - b) `remove()` ✓
 - c) `pop()`
5. What does `len([1, 2, 3])` return?
 - a) 2
 - b) 3 ✓
 - c) 4

Grading Rubric — Lab 05

Criterion	Weight	Not acquired (0)	Partial (0.5)	Acquired (1)
List creation and manipulation	20%	Cannot create a list	Basic operations	Complete mastery
List methods	20%	Uses no methods	Some methods	All methods
List algorithms	20%	No algorithm	Partial algorithm	Correct algorithm
List comprehension	20%	Does not know	Haphazard use	Appropriate use
Nested dictionaries	20%	Does not master	Partial use	Correct use

Session Schedule — 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome	5 min	5 min
Exercise 1 — List manipulation	15 min	20 min
Exercise 2 — Search	15 min	35 min
Exercise 3 — Grades with coefficients	20 min	55 min
Exercise 4 — Bubble sort	20 min	75 min
Exercise 5 — Filtering	15 min	90 min
Total	90 min	

Pedagogical Note ENS

Teaching transposition

This lab introduces Python lists as a fundamental data structure. Bubble sort (exercise 4) provides a concrete illustration of $O(n^2)$ algorithmic complexity seen in the theoretical course. In a secondary school classroom setting:

- Middle school (cycle 4): Use exercise 2 (search) to learn how to organize data. Ask students to look up a word in a paper dictionary, then compare with the computer algorithm.
- High school (SNT/NSI): Exercise 3 (grades with coefficients) is a real academic tracking tool.

Trainer advice

Explain carefully that list indices start at 0 (not 1). Use sticky notes on the board to visualize a list and insertion/deletion operations. For Seconde SNT students, limit exercises to simple lists (without negative indices or slicing) to avoid confusion.

Extension

Implement a binary search algorithm on a sorted list of words from the English dictionary. Compare the number of steps needed with sequential search.